CS3244 Progress Report
Group 6

Name: Bryan Kee
Group leader: Bryan Kee
Mentor's name: Ma Jianfei

The progress report is divided into two parts. The first part is the group report. It should be mostly an updated version of the project proposal, updated to reflect changes since the project was proposed. This part should be identical for each group member. The second part is an individual report and is graded individually.

Group Report

Dataset chosen and description Write in your own words (a few sentences) what this dataset is about.

The dataset used in this project is the STEG electricity fraud detection dataset, which contains electricity consumption and billing records from the Tunisian Company of Electricity and Gas. The dataset consists of two main components: a client-level dataset, where each instance corresponds to a customer with a binary fraud label, and an invoice-level dataset containing detailed billing records over time. Since each client may have multiple invoices, the data exhibits a one-to-many relationship.

To construct a machine learning dataset, invoice-level records are aggregated into client-level features that capture consumption behaviour over time, including statistical summaries and temporal patterns. The dataset is highly imbalanced, with fraudulent cases forming only a small proportion (approximately 5%) of all instances. This imbalance significantly impacts both model training and evaluation, and is a central challenge addressed in this project.

The dataset contains approximately 130,000 labelled instances, which is sufficiently large to support stable model training and evaluation. This scale allows us to obtain relatively low-variance estimates during cross-validation, reducing the need for excessively large numbers of folds. However, it also introduces computational considerations, particularly for models such as K-Nearest Neighbours and Kernel Support Vector Machine that scale poorly with dataset size.

Project Title

Machine Learning Approaches for Electricity Fraud Detection under Severe Class Imbalance

Motivation Explain why this project is interesting and important and the problem you aim to address.

Electricity fraud leads to substantial financial losses for utility providers and increases costs for legitimate consumers. Detecting such fraud is challenging because fraudulent behaviour is rare and often subtle, resulting in highly imbalanced datasets.

Traditional rule-based systems are limited in their ability to capture complex consumption patterns and often fail to generalise to unseen fraud behaviours. For example, a rule-based approach might flag customers whose electricity usage drops sharply or deviates significantly from historical averages. However, such rules are brittle as legitimate changes in consumption may trigger false positives, while more sophisticated fraudulent behaviour that mimics normal consumption patterns may go undetected. Furthermore, in highly imbalanced settings, these rules tend to be biased towards the majority class, leading to poor detection of rare fraudulent cases.

Machine learning methods offer a data-driven alternative by learning patterns directly from historical billing records and capturing more complex, non-linear relationships in the data. However, their effectiveness depends heavily on how class imbalance is handled. In this project, we focus not only on comparing different machine learning models, but also on understanding how imbalance handling techniques and evaluation strategies affect model performance. In particular, we investigate how choices such as resampling, model selection criteria, and decision thresholding interact in an imbalanced fraud detection setting.

General Approach A high-level description of the general approach you'll use to address the questions. Sketch out how you plan to run the necessary analysis and experiments.

The project follows a structured machine learning pipeline consisting of data preprocessing, feature engineering, model training, and evaluation.

First, invoice-level data is aggregated into client-level features that capture statistical and temporal characteristics of electricity consumption. Feature engineering includes both baseline statistical summaries and more advanced features informed by domain knowledge and feature selection techniques.

The dataset is split into training and test sets, with the test set held out for final evaluation. To ensure reproducibility and consistency across experiments, a unified experiment pipeline is implemented. This pipeline supports multiple models and enables systematic hyperparameter tuning using cross-validation.

Several machine learning models are considered, including Logistic Regression, Support Vector Machines, K-Nearest Neighbours, and Decision Trees. These models represent different classes of algorithms and allow comparison between linear and non-linear approaches.

To address class imbalance, resampling techniques such as SMOTE are used alongside model-based approaches such as class weighting (where supported). Class weighting adjusts the loss function to penalise misclassification of the minority class more heavily, while SMOTE generates synthetic minority samples to improve class representation. Importantly, resampling is applied only within training folds during cross-validation to avoid data leakage.

Evaluation Indicate how you will evaluate your project, i.e., how will you evaluate how your experiments have turned out?

Due to the severe class imbalance, accuracy is not an appropriate evaluation metric. Instead, model selection is performed using Precision-Recall AUC (PR-AUC), which focuses on performance of the positive (fraud) class under class imbalance without requiring a fixed classification threshold.

The evaluation framework follows a two-stage process:

1. **Model Selection (Threshold-independent):**
   Models are compared using PR-AUC obtained via cross-validation. This ensures fair comparison without bias from arbitrary threshold choices.
2. **Threshold Selection (Threshold-dependent):**
   After selecting the best model and hyperparameters, a decision threshold is determined using out-of-fold predictions on the training data. These predictions are generated from validation folds that are not seen during training, ensuring that threshold selection is unbiased and not influenced by overfitting or resampling artefacts.

Final evaluation is performed on a held-out test set using metrics including Precision, Recall, F1-score, and PR-AUC. Particular emphasis is placed on recall for the fraud class, while also considering the trade-off with precision.

Team Milestones

Milestone	Deadline
Phase 1: Planning & Baseline Development ~~Dataset exploration and feature engineering~~ ~~Research on imbalance handling techniques~~ ~~Implementation of the evaluation and experiment pipeline~~ ~~Implementation of baseline models~~ ~~Preliminary evaluation of model performance~~ ~~Internal discussion and preparation of the progress report~~	29 March (before progress report)
Phase 2: Model Improvement & Final Analysis ~~Refinement of feature engineering based on preliminary results~~ ~~Implementation of imbalance handling techniques~~ Hyperparameter tuning and model improvement Evaluation and comparison of model performance Exploring more complex models & ensemble methods Discussion of findings and consolidation of results Preparation of final report and analysis	19 April (before final report)

Team Progress

Write one or two paragraphs on progress towards the milestones scheduled before the progress report deadline. Highlight important decisions as well as difficulties encountered, and the changes that are made due to the difficulties.

The team has completed the initial phase of the project, including data preprocessing, baseline feature engineering, and the development of a unified experiment pipeline. Initial experiments showed that all models performed poorly on the positive class, and exhibited stable performance across validation folds, suggesting that they are underfitting the data and that high bias is the primary limitation. This is likely due to severe class imbalance and limited feature representation. Notably, tree-based models performed relatively better, indicating that non-linear methods may be more suitable for the task.

Based on these findings, the team refined its approach by improving feature engineering using domain insights and feature selection techniques, while also incorporating imbalance handling methods such as SMOTE and model-based class weighting. This allows us to compare data-level and algorithm-level approaches to handling imbalance, and to study how these methods interact with different models. A key challenge encountered was the interaction between resampling and threshold selection, where selecting thresholds on resampled data introduced bias. To address this, the evaluation pipeline was redesigned to use out-of-fold predictions for threshold selection, ensuring a more reliable and unbiased evaluation. The team is currently conducting a second round of experiments using the improved pipeline.

Individual Report

Individual Milestones

Highlight the milestones that you are responsible for.

Name	Milestone	Deadline
Bryan	Implement an appropriate experiment & evaluation pipeline	15 March (Sun)

Individual Progress

Write at least one paragraph (excluding tables and/or figures) on your progress for each of your milestones scheduled before the progress report deadline. Highlight important decisions as well as difficulties encountered, and the changes that are made due to the difficulties. Provide the results you have obtained, in tables and/or figures if appropriate, for your individual component. If you are the team leader, you should also write at least one paragraph about the efforts that the team have put towards ensuring that all the individual results can be compared in order to form a coherent final report.

Personal Contributions

My primary contribution was the design and implementation of a unified experiment pipeline to ensure consistent, reproducible, and comparable evaluation across all models. I set up a centralised GitHub repository and developed a modular pipeline architecture that standardised data loading, preprocessing, cross-validation, hyperparameter tuning, and result logging. To support collaboration, I abstracted model components, allowing team members to independently implement and tune their own models while remaining aligned with a shared evaluation framework. I also created initial scripts to aggregate and structure the raw data into a consistent format, reducing friction for the feature engineering process. A key design decision was the use of PR-AUC as the primary metric for model selection, given the highly imbalanced nature of the dataset. This avoided reliance on threshold-dependent metrics such as F1-score during hyperparameter tuning, which could otherwise produce misleading comparisons. The pipeline also included systematic logging of results to facilitate easy comparison and analysis across experiments.

Following team discussions and the introduction of resampling techniques such as SMOTE, I further refined the pipeline to address bias in threshold selection. The initial approach of selecting thresholds on the fully resampled training set was identified as problematic, as it could lead to overly optimistic performance estimates. To resolve this, I redesigned the evaluation framework to incorporate out-of-fold predictions, where thresholds are determined using validation predictions from cross-validation. This ensures that threshold selection is based on unseen data and better reflects real-world model behaviour, making the evaluation process more robust and reliable.

Team Contribution & Coordination

As team lead, I coordinated project progress by organising meetings, setting deadlines, and guiding discussions on experimental results. I encouraged deeper analysis by prompting team members to explain model behaviour and identify improvements, such as examining the curse of dimensionality in KNN and the limitations of linear models. These insights are explored in greater depth in the reports of team members responsible for model experimentation.

I also directed the initial feature engineering by reviewing prior work on the dataset and establishing a baseline feature set, followed by suggesting systematic methods such as mutual information for feature selection. In addition, I identified and resolved a data leakage issue in the initial resampling implementation, ensuring that all preprocessing steps were correctly applied within the cross-validation framework. To maintain consistency, I enforced a unified evaluation pipeline with aligned metrics and validation strategies across all team members.

In terms of time allocation, approximately 70–80% of my effort was spent on coordination, pipeline design, and integration, while the remaining 20–30% was devoted to implementation and experimentation to validate the pipeline and obtain preliminary comparisons.

Preliminary Baseline Results

Model	ROC AUC	PR AUC	F1 Score	Precision	Recall	PR AUC (cv mean)	PR AUC (cv std)
Decision Tree	0.826	0.283	0.358	0.333	0.386	0.271	0.00981
K Nearest Neighbours	0.809	0.223	0.149	0.081	0.946	0.207	0.00582
Logistic Regression	0.818	0.202	0.308	0.229	0.473	0.209	0.0117
Linear Support Vector Machine	0.811	0.216	0.323	0.236	0.510	0.224	0.0136

The baseline results were obtained using tuned models on a baseline feature set **without resampling or feature selection**. While all models perform better than a no-skill baseline, they show a consistent limitation in discriminating the positive class. This indicates that class imbalance and limited feature representation are key issues affecting performance across all models. Notably, the similarity between cross-validation and test PR-AUC suggests that the models are not overfitting, but are instead underfitting the data.

Among the models, the Decision Tree achieves the best overall balance between precision and recall, as reflected by its higher F1-score. However, since these limitations are observed across all models, the team has decided to prioritise addressing systemic issues rather than further model-specific tuning. Current efforts focus on improving feature quality and incorporating imbalance handling techniques, including both SMOTE and class weighting, into the pipeline. Depending on the outcomes, more expressive models such as ensemble methods (e.g. boosting-based tree models), which are effective at reducing bias, may be explored if underfitting remains the primary constraint.